Object-Oriented Programming

Luisa Daniela Correa Torres - 20242020048
Juan Pablo Rodríguez - 20251020072
Bryan Martinez Carreño - 20242020257

Workshop No. 4— Object-Oriented SOLID Principles

Universidad Distrital Francisco José de Caldas

November 28, 2025

1 Project Definition

1.1 Project Name

Prestige Eventos & Recepciones

1.2 Description

This project is for people planning a big party, like a wedding or a large birthday. Right now, planning something like this is hard because you have to call many different people (for the place, the food, and the decorations) and try to put all the pieces together yourself. This system makes it easy. It's a single website where you can see what dates are available, choose a party theme, pick the food you want, and add other services. You get to see the total price right away and book everything at once. The main goal is to take the stress out of planning a big event, making it a simple and clear process for everyone.

1.3 Main Features

The system guides you through planning your entire event in one place. You can start by choosing a fun theme for your party, like a classic wedding or a tropical birthday, which suggests matching decorations. Then, you'll pick the perfect food and drinks for your guests, from a formal dinner to a casual buffet. Finally, you can check the availability of your favorite venue, lock in your ideal date, and tell us how many people are coming to get everything officially booked. To make it easy, you can create a personal account to save your ideas and manage all your party details securely.

2 Requirements

2.1 Functional Requirements

- User registration and authentication: Customers can create an account and log in to manage their reservations.
- Date calendar: The user can check availability on an interactive calendar.
- Event booking: The client selects the date, type of party, number of guests, and additional services.(now `Reservation` coordinates with abstractions `IBookable` and `IPriced`)
- Modify: The user must be able to cancel or modify their reservation within two weeks.
- Support: The user should be able to communicate with us in the way that best suits them.
- Service Packages: The system must allow users to view and select predefined service packages that combine venue, theme, and catering options.
- Online Payments: The user must be able to securely complete online payments for their reservations and receive immediate payment confirmation.

- Automatic Notifications: The system must automatically generate and send email confirmations for both completed bookings and any subsequent modifications made to the reservation.

2.2 Non-Functional Requirements

- Ease of Use: The design must be simple and clear so that anyone, even without experience, can easily book their event.
- Mobile-Friendly: The website must work perfectly and look good on any device, like smartphones and tablets.
- Security: User data and payment information must be completely protected with modern security measures.
- Scalability: The system must work smoothly even when many people are using it at the same time without slowing down or crashing.
- Security: User data and payment information must be completely protected with modern security measures.
- Availability: The system must be working and available to users 24/7, especially on weekends when most people plan events.

3 User Stories

—

User Story 1

Title	Choose the Best Party Package
As a	client,
I want to	see and compare different pre-defined party packages (that include a theme, food, and decorations),
so that	I can easily choose the one that best fits my needs and vision without having to plan every detail myself.
Priority	High
Effort	5
Acceptance Criteria	• The main page must show a gallery of available party packages. • Each package must display a clear title, a main image, a short description, and a starting price. • I must be able to click on a package to see a detailed page with a full list of what's included (venue, food options, decoration style, etc.).

—

User Story 2

Title	Check Availability on a Calendar
As a	client,
I want to	see an interactive calendar that shows which dates are available for my chosen venue,
so that	I can quickly find and select a date that works for me and my guests.
Priority	High
Effort	3
Acceptance Criteria	• The calendar must be visually clear, with available dates marked differently from booked dates. • Clicking on an available date should allow me to proceed with the booking for that date. • If a date is unavailable, it should be clearly marked and not selectable.

—

User Story 3

Title	Modify or Cancel a Reservation
As a	customer,
I want to	cancel or make changes to my reservation (like the number of guests or food menu),
so that	I have the flexibility to adjust my plans if my needs change.
Priority	Medium
Effort	5
Acceptance Criteria	• From 'My Account,' I must see a list of my active reservations with an 'Edit' and 'Cancel' button. • I can only cancel or change my reservation up to 14 days before the event date. • After any change, the system must show me an updated summary and a new total price. • I must receive an email confirmation after any modification or cancellation.

—

User Story 4

Title	Create a Personal Account
As a	customer,
I want to	create a personal account using my email and a password,
so that	I can save my preferences, see my booking history, and make future reservations faster.
Priority	High
Effort	3
Acceptance Criteria	• The registration form must only ask for my full name, email, and a secure password. • I must receive a verification email to activate my account. • Once logged in, I must have a "My Account" page where I can see my profile and past bookings.

—

User Story 5

Title	Contact the Organizers Easily
As a	customer,
I want to	have multiple, clear ways to contact customer support (like a contact form and a phone number),
so that	I can get help in the way that is most convenient for me if I have a question or problem.
Priority	Low
Effort	2
Acceptance Criteria	• The website's footer and contact page must display a customer service phone number and email address. • A contact form must be available that sends my message directly to the support team. • After submitting the form, I must see an on-screen confirmation that my message was sent.

—-

4 CRC Cards

Class: Venue	**Class: User**
Responsibilities: • Save place info • Check dates • Verify capacity • Manage calendar	**Responsibilities:** • Create accounts • Save user data • Manage preferences
Collaborators: • Reservation • Calendar	**Collaborators:** • Reservation • Payment

Class: Calendar	**Class: Payment**
Responsibilities: • Show dates • Block dates • Date selection	**Responsibilities:** • Process payments • Payment methods • Generate receipts • Handle refunds
Collaborators: • Venue • Reservation	**Collaborators:** • Reservation • User

Class: Service	**Class: Reservation**
Responsibilities: • Manage services • Save prices • Check availability • Calculate cost	**Responsibilities:** • Create reservations • Calculate total • Change status • Manage dates
Collaborators: • Reservation	**Collaborators:** • User • Venue • Service • Payment

Interface: IPayment	
Responsibilities: • Define a standard method to process payments abstractly. • Allow multiple implementations for different payment methods.	**Collaborators:** • Reservation

Interface: INotification	
Responsibilities: • Define an abstraction for sending notifications to users. • Ensure flexible communication mechanisms (email, SMS, etc.).	**Collaborators:** • User • Reservation

Abstract Class: AbstractService	
Responsibilities: • Define common attributes and behaviors for all services. • Allow extensions without modifying existing service logic.	**Collaborators:** • Service • Reservation

Class: CateringService (extends AbstractService)	
Responsibilities: • Handle catering-specific booking and availability logic. • Provide custom pricing for catering events.	**Collaborators:** • Reservation • Venue

Interface: IBookable	
Responsibilities: • Define methods to verify and book availability for a service or venue.	**Collaborators:** • Reservation • Service

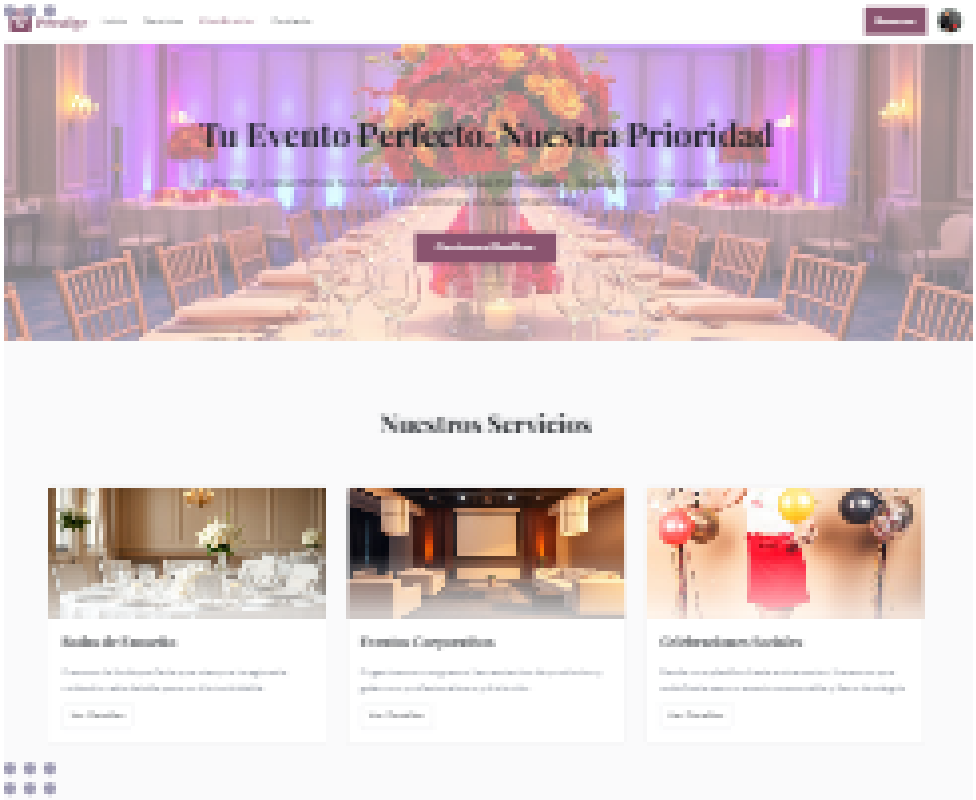
4.1 Solid -Focused Implementation

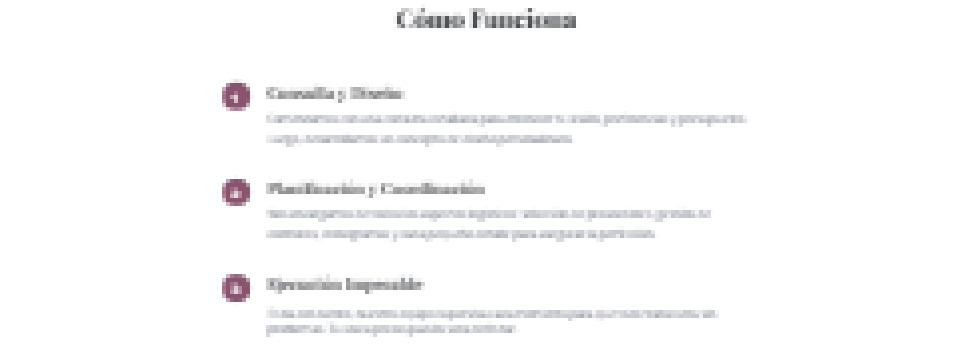
- Single Responsibility : payment no longer sends confirmation; NotificationService now sends the confirmation
- Open/close : created interfaces for new services(catering, decoration , audio)
- Liskov substitution : we introduce AbstractService and concrete Service
- interface Segregation :IBookable, IPriced, IPaymentProcessor, INotification replace thelarge interfaces.
- Dependency Inversion : high level modules depend on abstraction IPaymentProcessor, INotification

b

5 Mockups

User Interface Designs





Explora Nuestras Delicias Culinarias

Desde exquisitos platillos para eventos íntimos hasta banquetes especializados, Prestige Catering ofrece opciones para cada ocasión. Descubre nuestra variada oferta y personaliza tu experiencia gastronómica.

Solicita una Cotización

Nuestras Categorías de Menú

Todos | Entradas | Platos Principales | Postres | Bebidas | Buffet | Personalizados

Ensalada Caprese Fresca

Una deliciosa mezcla de tomates cherry maduros, mozzarella fresca y hojas de albahaca, aderezada con glaseado balsámico. Perfecta como entrada ligera y refrescante.

Vegetariano | Saludable

COP $25.000

Solicitar Información

Filete Mignon con Salsa de Champiñones

Cortes premium de filete de res cocinados a la perfección, servidos con una rica salsa cremosa de champiñones, acompañados de vegetales frescos y puré de papa.

Carne | Gourmet

COP $70.000

Solicitar Información

Tarta de Limón con Merengue Suizo

Una exquisita tarta con una base de galleta crujiente, relleno de crema de limón suave y cubierta con un merengue suizo tostado. Ideal para finalizar cualquier comida.

Postre | Dulce

COP $50.000

Solicitar Información

Paella Valenciana Tradicional

Auténtica paella española con pollo, conejo, judías verdes y garrofón, cocinada lentamente para lograr el sabor y la textura perfectos. Un clásico que no debería faltar en tus eventos.

Internacional | Compartir

COP $60.000

Solicitar Información

Tabla de Quesos Artesanales

Una cuidadosa selección de quesos madurados e importados, acompañada de frutas frescas, frutos secos, mermeladas y pan artesanal. Perfecta para compartir y disfrutar en buena compañía.

Aperitivo | Gourmet

COP $50.000

Solicitar Información

Brochetas de Camarones al Ajillo

Jugosos camarones marinados en ajo, limón y hierbas, asados a la perfección y servidos en una porción rústica y aromática. Un entrante elegante y lleno de sabor.

Mariscos | Ligero

COP $65.000

Solicitar Información

Lasagna Casera Boloñesa

Capas de pasta fresca, una rica salsa boloñesa hecha con carne de res y tomate, y una cremosa bechamel, horneadas hasta obtener un sabor perfecto. Un plato reconfortante y delicioso.

Italiano | Comfort

COP $45.000

Solicitar Información

Mesa de Postres Variados

Una selección personalizable de irresistibles postres que incluye macarons, tortas de chocolate, mini cheesecakes y tortas italianas de chocolate. Ideal para eventos grandes.

Variado | Celebración

COP $90.000

Solicitar Información

Bebidas Refrescantes Naturales

Una selección de bebidas naturales, incluyendo limonadas y aguas frescas de temporada. Perfectas para acompañar cualquier menú y mantener a los invitados hidratados.

Naturales | Frescas

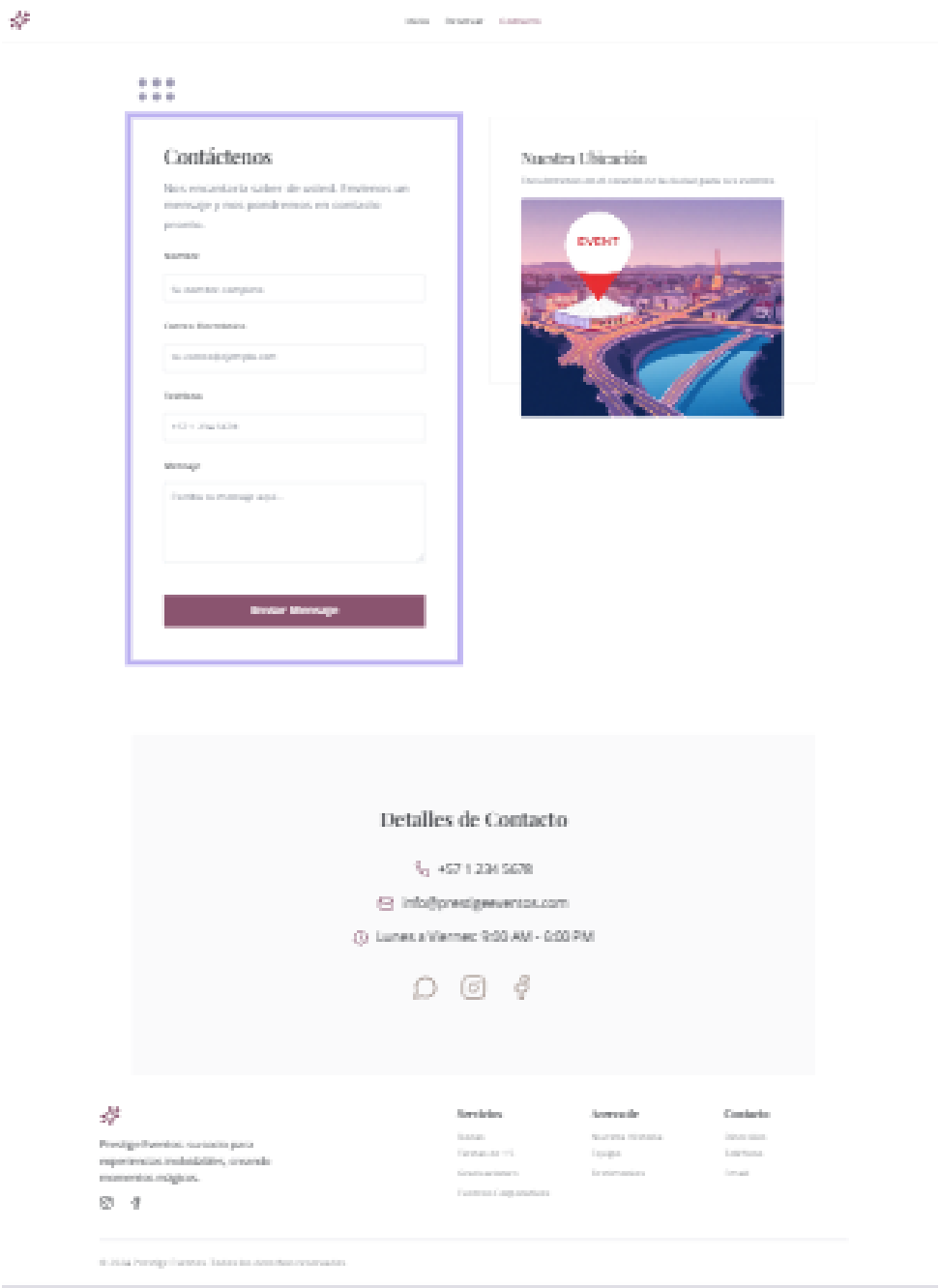
COP $10.000

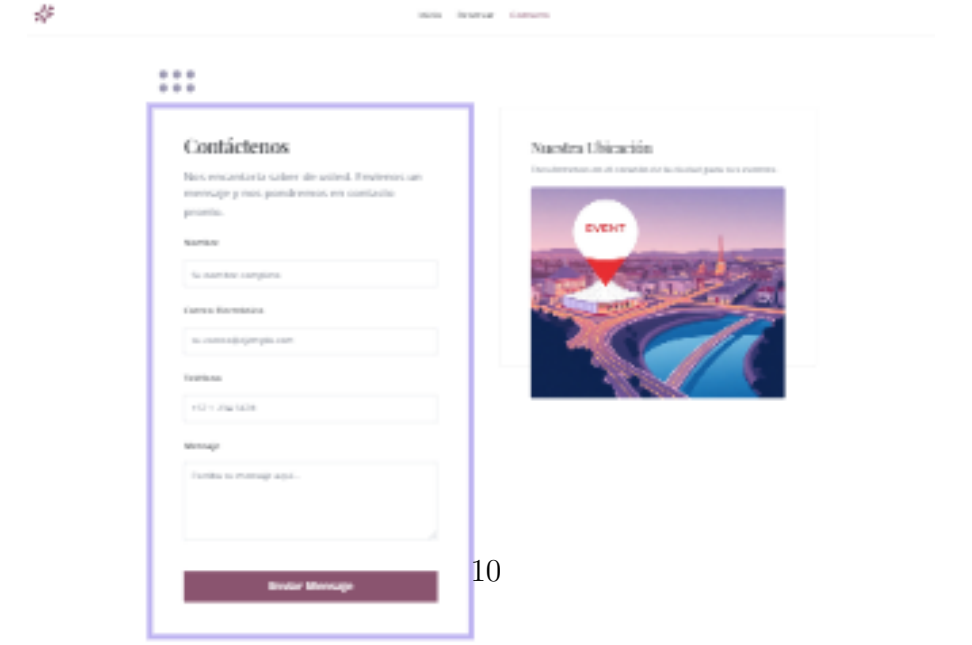
Solicitar Información

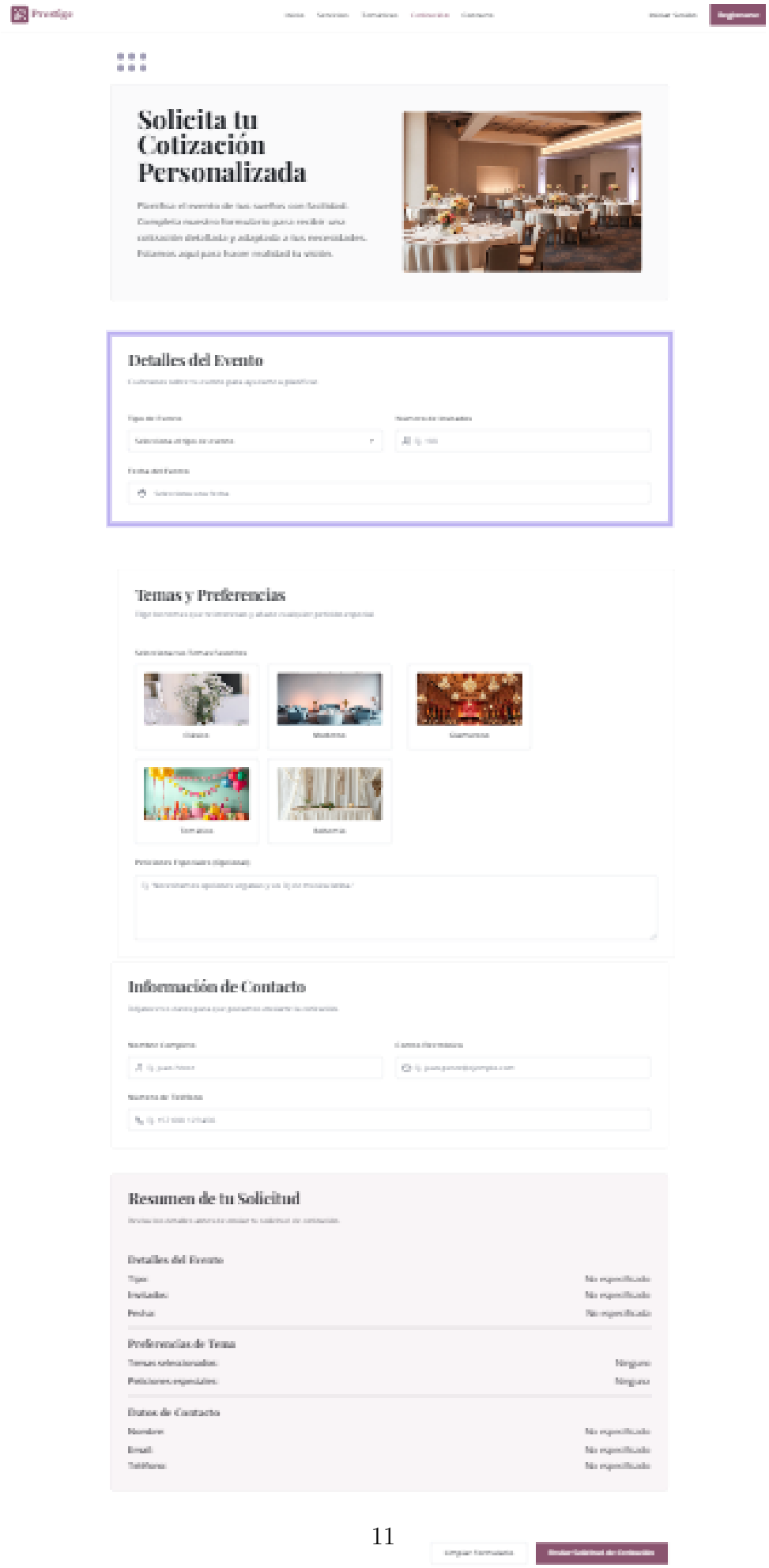
Listo para planificar tu evento?

Contáctanos hoy para diseñar un menú perfecto que deleite a tus invitados. Nuestro equipo está listo para asesorarte en cada detalle.

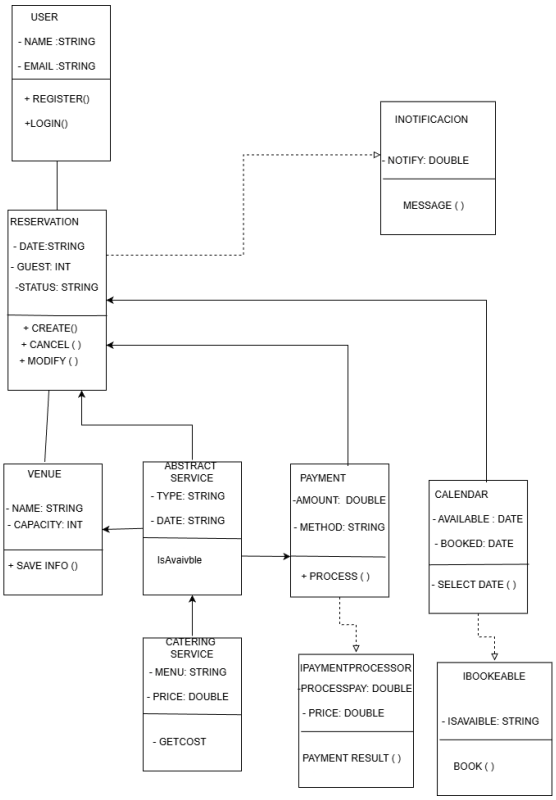
Contáctanos Ahora

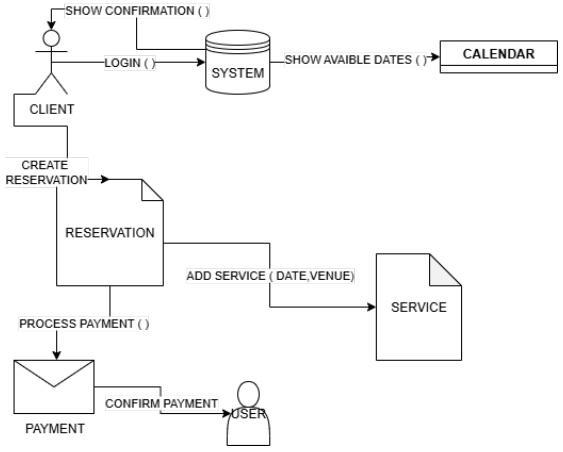

Contáctenos
Nuestra Ubicación
EVENT
Detalles de Contacto
+57 1 234 5678
info@prestigeventos.com
Lunes a Viernes: 9:00 AM - 6:00 PM

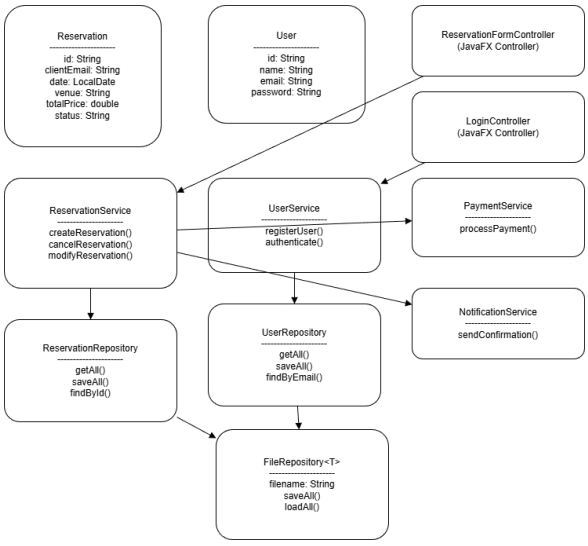

Contáctenos
Nuestra Ubicación
EVENT


Prestige
Solicita tu Cotización Personalizada
Detalles del Evento
Temas y Preferencias
Información de Contacto
Resumen de tu Solicitud

6 UML DIAGRAMS







7 Java Implementation - Applying SOLID Principles

This section show how the original system was improved using the SOLID principles, based on the existing base class. Each principle is explained, along with the changes made and the specific classes where they were applied.

7.1 Single Responsability Principle (SRP)

Original Issue: The Payment class had two responsibilities: processing payments and sending confirmations.
Improvement: A new class called NotificationService was created to handle all message sending and confirmations. Now, Payment only focuses on payment logic.

```
// New class to apply SRP
public class NotificationService {
    public void sendConfirmation(Client client, String message) {
        System.out.println("Sending confirmation to " + client.getName() + ":
        " + message);
    }
}
```

```
// Modified Payment.java
public class Payment {
    private double amount;
    private boolean isProcessed;
    private NotificationService notificationService;

    public Payment(double amount, NotificationService notificationService) {
        this.amount = amount;
        this.notificationService = notificationService;
        this.isProcessed = false;
    }

    public void processPayment() {
        this.isProcessed = true;
        System.out.println("Payment of $" + amount + " processed");
    }

    public void confirmPayment(Client client) {
        if (isProcessed)
            notificationService.sendConfirmation(client, "Payment confirmed!");
        else
            System.out.println("Payment not processed yet");
    }
}

public class Client {
    private String name, email, password;
    public Client(String name, String email, String password) {
```

```
        this.name = name; this.email = email; this.password = password;
    }
    public boolean login(String email, String password) {
        return this.email.equals(email) && this.password.equals(password);
    }
    public void showConfirmation(String message) {
        System.out.println("Confirmation: " + message);
    }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}
```

Where applied:
Responsibility for notifications was moved to a new class (NotificationService).
Payment now has a single, focused responsibility.

7.2 Open/Closed Principle (OCP)

Original Issue: The Service class was static and not easily extendable. Adding new types of services (like decoration or audio) required modifying the base code.
Improvement: Created an AbstractService class and new subclasses (e.g., CateringService) that extend it. Now, new services can be added without changing the base class.

```
//Abstract base class
public abstract class AbstractService {
    protected String name;
    protected double price;

    public AbstractService(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public abstract void addService(String date, String venue);
}

//Example subclass applying OCP
public class CateringService extends AbstractService {
    public CateringService(double price) {
        super("Catering", price);
    }

    @Override
    public void addService(String date, String venue) {
        System.out.println("Catering service booked for " + date + " at " + venue);
    }
}
```

Where applied:
The old Service class was replaced with an extensible hierarchy.
Future services can be added by subclassing, not by editing existing files.

7.3 Liskov Sustitution Principle (LSP)

Implementation: Because all services now inherit from AbstractService, any subclass can be used interchangeably where the base type is expected.

```
//Example in Main.java
AbstractService service = new CateringService(500.0);
service.addService("2025-11-20", "Main Hall");
```

Where applied:
CateringService can replace AbstractService without issues.
This maintains consistent behavior and supports polymorphism.

7.4 Interface Segregation Principle (ISP)

Original Issue: There were no interfaces; classes were highly coupled and dependent on each other.
Improvement: Created smaller, more specific interfaces for different functionalities.

```
    //New small interfaces
public interface IBookable {
    boolean checkAvailability(String date);
    void book(String date);
}

public interface IPriced {
    double calculatePrice();
}

public interface IPaymentProcessor {
    void process(double amount);
}

public interface INotification {
    void send(String message);
}
```

Where applied:
Each class now implements only the interfaces it needs.
Prevents unnecessary dependencies and improves modularity.

7.5 Dependency Inversion Principle (DIP)

Original Issue: The Main class directly instantiated concrete classes like Payment and Calendar, creating tight coupling. **Improvement:** Created interfaces for key behaviors (IPaymentProcessor, INotification) and updated classes to depend on these abstractions instead of specific implementations.

```
    // Implementation example for DIP
public class OnlinePaymentProcessor implements IPaymentProcessor {
    @Override
    public void process(double amount) {
        System.out.println("Processing online payment of $" + amount);
    }
}

// New class depending on abstraction
public class PaymentService {
    private IPaymentProcessor processor;

    public PaymentService(IPaymentProcessor processor) {
        this.processor = processor;
    }

    public void executePayment(double amount) {
        processor.process(amount);
    }
}
```

Where applied:
PaymentService depends on the abstraction IPaymentProcessor.
Enables easy replacement of payment methods without changing the high-level logic.

8 LAYERED ARCHITECTURE

To complete our project, we implement the layered architecture, the goal of this implementation being high cohesion, separation of responsibilities, the layers will be communicated in a top-down system

8.0.1 PRESENTATION LAYERS

This layer manages all user interactions and graphics
RESPONSIBILITIES

- Display screen(login,register, reservation, calendar)
- Collect user inputs
- Show confirmation or error messages

```
public void onCreateReservation() {
    String email = emailField.getText();
    LocalDate date = datePicker.getValue();
    String venue = venueField.getText();

    reservationService.createReservation(email, date, venue, 500.0);

    Alert alert = new Alert(Alert.AlertType.INFORMATION, "Reservation created!");
    alert.showAndWait();
}
```

8.0.2 SERVICES LAYER

This layer contains the application logic and enforces all the rules of the system.

RESPONSIBILITIES

- Validate bookings and availability
- Create, modify, and cancel reservations
- Notify users after changes

8.0.3 DATA LAYER

This layer simulates a database using local file storage **RESPONSIBILITIES**

- Load and save reservations and users
- Provide repository methods for the business layer

```
public void saveAll(List<T> items) throws IOException {
    ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(filename));
    out.writeObject(items);
    out.close();
}

public List<T> loadAll() throws an exception {
    File file = new File(filename);
    if (!file.exists()) return new ArrayList<>();

    ObjectInputStream in = new ObjectInputStream(new FileInputStream(filename));
    List<T> data = (List<T>) in.readObject();
    in.close();
    return data;
}
```

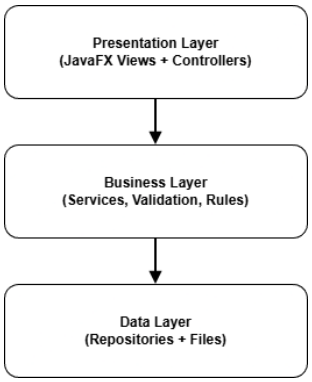


Figure 2: Layers interaction

9 JAVAFX IN THE PROJECT

This workshop requires creating a simple but functional JavaFX interface that demonstrates the core user interactions of the system

9.0.1 Implemented screens

- LoginView – allows a user to authenticate.
- MainView – navigation window showing the core system options.
- ReservationForm – form to create a new reservation.
- PackageListView – list of service packages.

10 STORAGE IMPLEMENTATION

10.0.1 REQUIREMENTS IMPLEMENT

- Saving all reservations to a file after each change.
- Loading all existing data when the program starts.
- Handling missing files by automatically creating new ones.

10.0.2 RESERVATION REPOSITORY

```
public class ReservationRepository {

        private final FileRepository<Reservation> repository =
           new FileRepository<>("reservations.dat");

       public List<Reservation> getAll() throws an exception {
       return repository.loadAll();
   }
      public void saveAll(List<Reservation> reservations) throws
      Exception {
    repository.saveAll(reservations); }
    }
```

10.1 REFLECTION

During this final phase, the main challenge was integrating previous SOLID-based designs into a fully layered architecture while ensuring minimal coupling between UI, business logic, and persistence. JavaFX required adapting our controllers, we have problems with javaFx because we don't have so much strength in the topic but we do some advances with using javaFx.